CS 550 Operating Systems Spring 2026

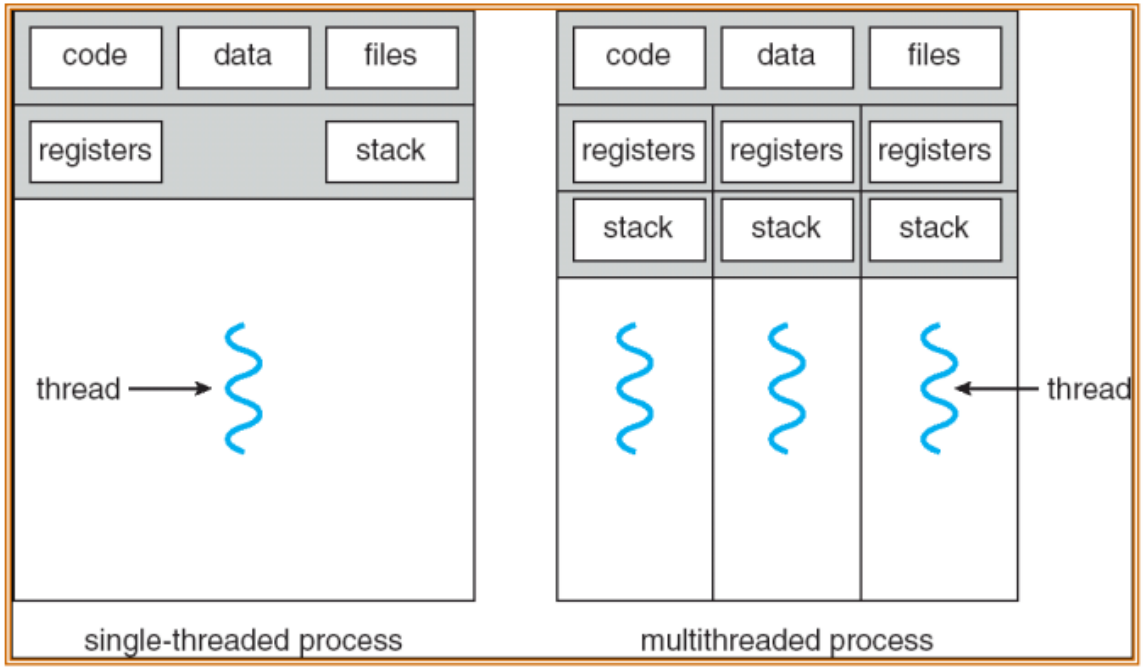
Threads intro and APIs

Threads

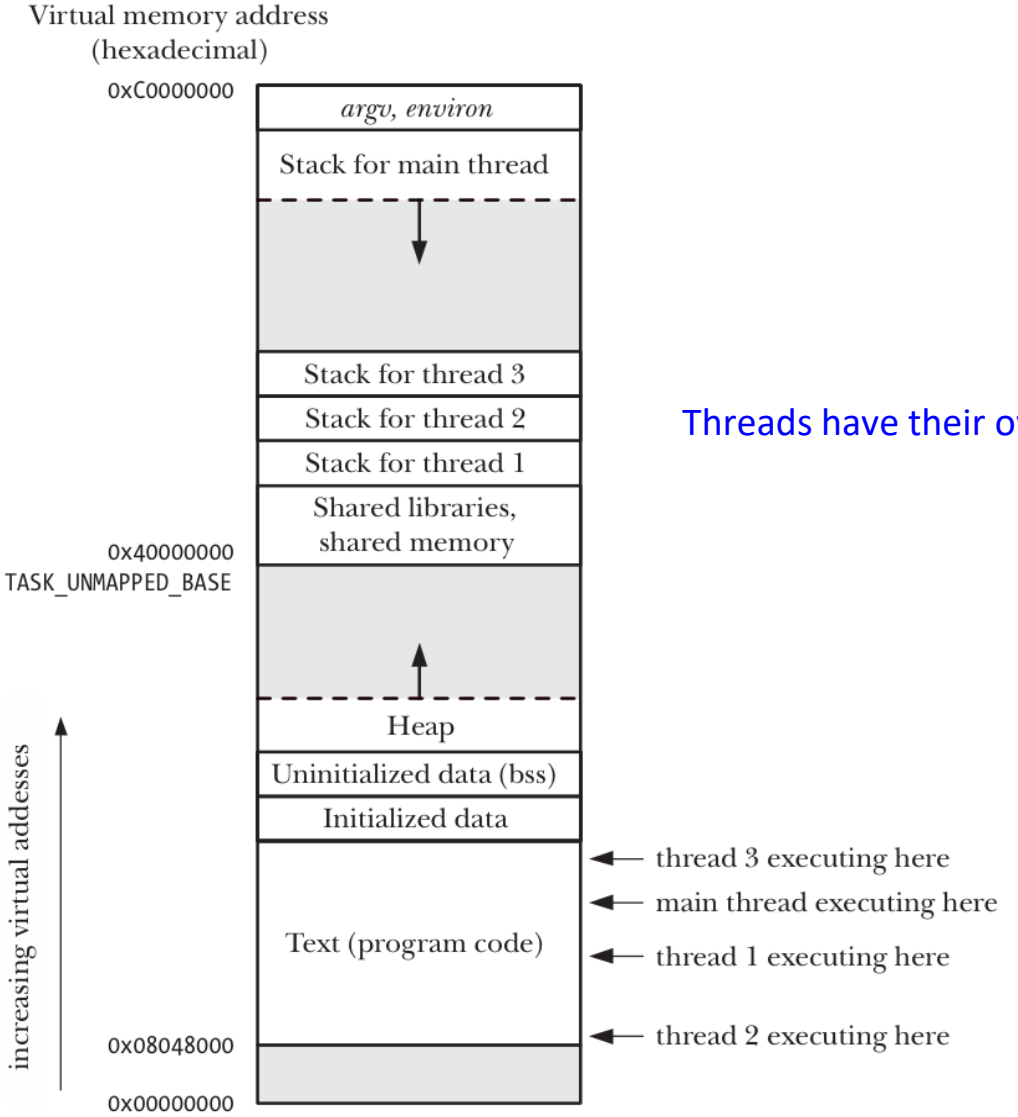
- **Threads**: separate streams of executions that share an address space.
  - Allows one process to have multiple point of executions, can potentially use multiple CPUs
- Thread control block (TCB)
  - Program counter (EIP on x86)
  - Other registers
  - Stack
- Very similar to processes, but different

Threads

- Threads within the same process
  - share the same code, data, and open files.
  - have their own set of register values and stacks.



Four threads executing in a process (Linux/x86-32)



Threads have their own stacks.

Threads share the same program code, but have their own PCs.

Threads vs. Processes

- Advantages of threads over processes?
  - Easy data sharing
    - Parent and child processes don't share memory, some form of IPC is required.
    - Global memory are shared among threads in a process
  - Faster creation speed
    - Many of the attributes that must be duplicated in a child created by `fork()` are instead shared between threads.
    - Duplication of page tables is not needed with thread creation.

Threads vs. Processes

- Disadvantages of threads over processes
  - Race conditions between threads.
  - A bug in one thread (e.g., modifying memory via an incorrect pointer) can damage all of the threads in the process (because global memory are shared).
  - Each thread is competing for use of the finite virtual address space of the host process.
    - For example, stack space shortage when there are a large number of threads, each of which is demanding a large stack space

POSIX thread API: pthread

- Implementations of the API are available on many Unix-like POSIX-conformant operating systems.
- There are around 100 Pthreads procedures, all prefixed "pthread_" and they can be categorized into four groups:
  - Thread management - creating, joining threads etc.
  - Mutexes
  - Condition variables
  - Synchronization between threads using read/write locks, semaphores, and barriers

Pthread API: thread identification

- A thread ID is represented by the pthread_t data type.
  - On many implementations the pthread_t type is represented using integers (e.g., unsigned long in Linux).
  - Different from the pid_t type, implementations are allowed to use a structure to represent the pthread_t data type.
  - Therefore, portable applications can't treat the pthread_t type as integer ➔ we need a function to compare thread IDs, instead of using "==" operator.

```
include <pthread.h>

int pthread_equal(pthread_t t1, pthread_t t2);
                                Returns nonzero value if t1 and t2 are equal, otherwise 0
```

Pthread API: process identification

- A thread can obtain its own thread ID by calling the pthread_self() function.

```
include <pthread.h>

pthread_t pthread_self(void);
```

Returns the thread ID of the calling thread

- Why do threads need to know their own thread IDs?
  - Various pthreads functions use thread IDs to identify the thread on which they are to act.
  - In some applications, it can be useful to tag dynamic data structures with the ID of a particular thread. This can serve to identify the thread that created or “owns” a data structure.
  - Both of the above come from the fact that threads within a process share the heap and the data segment

Pthread API: thread creation

- The traditional UNIX process model supports only one thread of control per process.
  - Conceptually, this is the same as a threads-based model whereby each process is made up of only one thread.
- With pthreads, when a program runs, it also starts out as a single process with a single thread of control.
  - As the program runs, its behavior should be indistinguishable from the traditional process, until it creates more threads of control.

Pthread API: thread creation

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start)(void *), void *arg);
```

Returns 0 on success, or a positive error number on error

- The *thread* argument set is to the thread ID of the newly created thread before `pthread_create()` returns.
- The *attr* argument is a pointer to a `pthread_attr_t` object that specifies various attributes for the new thread.
  - If *attr* is specified as NULL, then the thread is created with various default attributes

Pthread API: thread creation

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start)(void *), void *arg);
```

Returns 0 on success, or a positive error number on error

- The new thread commences execution by calling the function identified by *start*, with the argument *arg* (i.e., run as *start*(*arg*)).
  - *start* is a pointer to a function that takes a void pointer as input, and returns a void pointer.
  - *arg* points to a global or heap variable, but it can also be specified as NULL.

```
pthread_t ntid;

void
printids(const char *s)
{
    pid_t       pid;
    pthread_t   tid;

    pid = getpid();
    tid = pthread_self();
    printf("%s pid %lu tid %lu (0x%lx)\n", s, (unsigned long)pid,
      (unsigned long)tid, (unsigned long)tid);
}

void *
thr_fn(void *arg)
{
    printids("new thread: ");
    return((void *)0);
}

int
main(void)
{
    int     err;

    err = pthread_create(&ntid, NULL, thr_fn, NULL);
    if (err != 0)
        err_exit(err, "can't create thread");
    printids("main thread:");
    sleep(1);
    exit(0);
}
```

Pthread API: thread termination

- If any thread within a process calls `exit()` or `_exit()`, or the main thread performs a return in the `main()` function, then the entire process terminates.
- A single thread can exit in three ways, thereby stopping its flow of control, without terminating the entire process.
  - The thread calls `pthread_exit(void *)`.
  - The thread returns from the start routine. The return value is the thread's exit status.
  - The thread can be canceled by another thread in the same process.

Pthread API: thread termination

```
include <pthread.h>

void pthread_exit(void *retval);
```

- Calling `pthread_exit()` is equivalent to performing a return in the thread's start routine.
  - The difference that `pthread_exit()` can be called from any function that has been called by the thread's start function.
- The *`retval`* argument is a void pointer, similar to the single argument passed to the start routine.
  - This pointer is available to other threads in the process by calling the `pthread_join()` function.
  - The relation between `pthread_exit()` and `pthread_join()` is similar to `exit()` and `wait()` for process.

Pthread API: joining a terminated thread

```
include <pthread.h>

int pthread_join(pthread_t thread, void **retval);
```

Returns 0 on success, or a positive error number on error

- The `pthread_join()` function waits for the thread identified by *`thread`* to terminate. This operation is termed ***joining***.
- The calling thread will block until the specified thread calls `pthread_exit()`, returns from its start routine, or is canceled.
  - If the thread calls `pthread_exit()`, *`retval`* here points to the *`retval`* argument of `pthread_exit()`.
  - If the thread simply returned from its start routine, *`retval`* points to the return value of the start routine (which is also a void pointer).
  - If the thread was canceled, the memory location specified by *`retval`* is set to `PTHREAD_CANCELED`.

```c
void *
thr_fn1(void *arg)
{
    printf("thread 1 returning\n");
    return((void *)1);
}
```

```c
void *
thr_fn2(void *arg)
{
    printf("thread 2 exiting\n");
    pthread_exit((void *)2);
}
```

```c
int
main(void)
{
    int         err;
    pthread_t   tid1, tid2;
    void        *tret;

    err = pthread_create(&tid1, NULL, thr_fn1, NULL);
    if (err != 0)
        err_exit(err, "can't create thread 1");
    err = pthread_create(&tid2, NULL, thr_fn2, NULL);
    if (err != 0)
        err_exit(err, "can't create thread 2");
    err = pthread_join(tid1, &tret);
    if (err != 0)
        err_exit(err, "can't join with thread 1");
    printf("thread 1 exit code %ld\n", (long)tret);
    err = pthread_join(tid2, &tret);
    if (err != 0)
        err_exit(err, "can't join with thread 2");
    printf("thread 2 exit code %ld\n", (long)tret);
    exit(0);
}
```

```
$ ./a.out
thread 1 returning
thread 2 exiting
thread 1 exit code 1
thread 2 exit code 2
```

The typeless pointers in Pthread APIs

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start)(void *), void *arg);
                                    Returns 0 on success, or a positive error number on error
```

```
include <pthread.h>

void pthread_exit(void *retval);
```

- The typeless pointer returned from a thread's *start* function, or passed to `pthread_create()` and `pthread_exit()` can be used to pass more than a single value.
  - The pointer can be used to pass the address of a structure containing more complex information.

The typeless pointers in Pthread APIs

```
1   #include <stdio.h>
2   #include <pthread.h>
3   #include <assert.h>
4   #include <stdlib.h>
5
6   typedef struct __myarg_t {
7       int a;
8       int b;
9   } myarg_t;
10
11  typedef struct __myret_t {
12      int x;
13      int y;
14  } myret_t;
15
16  void *mythread(void *arg) {
17      myarg_t *m = (myarg_t *) arg;
18      printf("%d %d\n", m->a, m->b);
19      myret_t *r = Malloc(sizeof(myret_t));
20      r->x = 1;
21      r->y = 2;
22      return (void *) r;
23  }
```

```
25  int
26  main(int argc, char *argv[]) {
27      pthread_t p;
28      myret_t *m;
29
30      myarg_t args = {10, 20};
31      Pthread_create(&p, NULL, mythread, &args);
32      Pthread_join(p, (void **) &m);
33      printf("returned %d %d\n", m->x, m->y);
34      free(m);
35      return 0;
36  }
```

Another example …

```c
#include <stdio.h>
#include <pthread.h>
#include <assert.h>
#include <stdlib.h>

typedef struct __myarg_t {
    int a;
    int b;
} myarg_t;

typedef struct __myret_t {
    int x;
    int y;
} myret_t;
```

```c
void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    myret_t r;
    r.x = 1;
    r.y = 2;
    return (void *) &r;
}
```

Return addresses on stack is BAD!

Why?

```c
int main(int argc, char *argv[]) {
    pthread_t p;
    int rc, m;
    Pthread_create(&p, NULL, mythread, (void *) 100);
    Pthread_join(p, (void **) &m);
    printf("returned %d\n", m);
    return 0;
}
```

The typeless pointers in Pthread APIs

- Packing and unpacking arguments are not always necessary
  - When creating a thread with no arguments
    - Pass NULL as "arg" in `pthread_create()`.
    - Pass NULL into `pthread_join()` if we don't care about exit/return value of the thread
  - When just passing a single value to the thread created

Passing a single value to the thread

```
void *mythread(void *arg) {
    int m = (int) arg;
    printf("%d\n", m);
    return (void *) (arg + 1);
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc, m;
    Pthread_create(&p, NULL, mythread, (void *) 100);
    Pthread_join(p, (void **) &m);
    printf("returned %d\n", m);
    return 0;
}
```

```
int idata = 111;          /* Allocated in data segment */

int  main(int argc, char *argv[])
{
   int istack = 222;          /* Allocated in stack segment */
   pid_t childPid;

   childPid = fork();
   if (childPid == -1) {exit(-1);}
   else if (childPid == 0) {idata *= 3; istack *= 3;} /* Child Process: modify data */
   else {sleep(3)} /* Parent process: give child a chance to execute */

   /* Both parent and child come here */
   printf("PID=%ld %s idata=%d istack=%d\n", (long) getpid(), (childPid == 0) ? "(child) " :
"(parent)", idata, istack);

   exit(0);
}
```

```
$ ./t_fork
PID=28557 (child)  idata=333 istack=666
PID=28556 (parent) idata=111 istack=222
```

```
int idata = 111;          /* Allocated in data segment */

void *mythread(void *arg) {
   idata *= 3;
   istack *= 3;
   return (void *) 0;
}

int  main(int argc, char *argv[])
{
   int istack = 222;            /* Allocated in stack segment */
   pthread_t tid;

   pthread_create(&tid, NULL, mythread, NULL);

   sleep(1);


   printf("idata=%d istack=%d ", idata, istack);
   exit(0);
}
```

What is the outcome of this program?

```
int idata = 111;          /* Allocated in data segment */

void *mythread(void *arg) {
   idata *= 3;
   istack *= 3;      Undefined variable (each thread has its own stack)
   return (void *) 0;
}

int  main(int argc, char *argv[])
{
   int istack = 222;          /* Allocated in stack segment */
   pthread_t tid;

   pthread_create(&tid, NULL, mythread, NULL);

   sleep(1);    // Without it, it's not guaranteed that the following print in the main thread
                //  happens after the new thread exits

   printf("idata=%d istack=%d ", idata, istack);
   exit(0);       ?
}
```

idata=333 istack=222

Race condition between threads

```
void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");
    rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
    rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
    // join waits for the threads to finish
    rc = pthread_join(p1, NULL); assert(rc == 0);
    rc = pthread_join(p2, NULL); assert(rc == 0);
    printf("main: end\n");
    return 0;
}
```

Execution sequence 1:

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
waits for T1		
	runs	
	prints "A"	
	returns	
waits for T2		
		runs
		prints "B"
		returns
prints "main: end"		

Execution sequence 2:

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
	runs	
	prints "A"	
	returns	
creates Thread 2		
		runs
		prints "B"
		returns
waits for T1		
returns immediately; T1 is done		
waits for T2		
returns immediately; T2 is done		
prints "main: end"		

Execution sequence 3:

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
		runs
		prints "B"
		returns
waits for T1		
	runs	
	prints "A"	
	returns	
waits for T2		
returns immediately; T2 is done		
prints "main: end"		

Threads race condition: sharing data

```
static volatile int counter = 0;

void *
mythread(void *arg)
{
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[])
{
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");

    // join waits for the threads to finish
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n", counter);
    return 0;
}
```

What we expect …

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

But actually …

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```

Source of the problem: uncontrolled scheduling

```
counter = counter + 1;
```

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

OS	Thread 1	Thread 2	PC (after instruction)	%eax	counter
	before critical section		100	0	50
	mov 0x8049a1c, %eax		105	**50**	50
	add $0x1, %eax		108	**51**	50
interrupt					
save T1's state					
restore T2's state			100	0	50
		mov 0x8049a1c, %eax	105	**50**	50
		add $0x1, %eax	108	**51**	50
		mov %eax, 0x8049a1c	113	51	**51**
interrupt					
save T2's state					
restore T1's state			108	51	50
	mov %eax, 0x8049a1c		113	51	**51**

Key concurrency terms

- **Critical section**: a piece of code that accesses a shared resource, usually a variable or data structure.
- **Race condition** arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure.
- An **indeterminate** program consists of one or more race conditions; the output of the program varies from run to run, depending on which threads ran when. The outcome is thus not **deterministic**, something we usually expect from computer systems.
- To avoid these problems, threads should use some kind of **mutual exclusion** primitives; doing so guarantees that only a single thread ever enters a critical section, thus avoiding races, and resulting in deterministic program outputs.